```
        var listViewModel = new ToDoListModel();    ❶
        return View(listViewModel);                 ❷
    }
    public IActionResult View(int id)
    {
        var viewModel = new ViewToDoModel();
        return View("ViewToDo", viewModel);         ❸
    }

}
```

❶ Creates an instance of the view model to pass to the Razor view

❷ The view model is passed as an argument to View.

❸ You can provide the view template name at the same time as the view model.

Once the Razor view template has been located, the view is rendered using the Razor syntax you learned about in chapters 17 and 18. You can use all the features you've already seen—layouts, partial views, _ViewImports, and _ViewStart, for example. From the point of view of the Razor view, there's no difference between a Razor Pages view and an MVC Razor view.

Now you've had a brief overview of an MVC application, we can look in more depth about when to choose MVC controllers over Razor Pages.

19.5 Choosing between Razor Pages and MVC controllers

Throughout this book, I have said that you should generally choose Razor Pages for server-rendered applications instead of using MVC controllers. In this section I show the difference between Razor Pages and MVC controllers from a project structure point of view and defend my reasoning. I also describe the cases where MVC controllers are a good choice.